บทที่ 1

ความรู้เบื้องต้นเกี่ยวกับ React Native Expo และการเตรียมสภาพแวดล้อม

ยินดีต้อนรับสู่หลักสูตรพัฒนาแอปพลิเคชันมือถือด้วย **React Native และ Expo SDK 54** ในบทนี้เราจะเรียนรู้แนวคิดพื้นฐาน ติดตั้งเครื่องมือ และเริ่มต้นรันแอปพลิเคชันแรกของคุณ

1.1 ทำความรู้จัก React Native และ Expo

React Native คืออะไร?

React Native คือโอเพนซอร์สเฟรมเวิร์กที่พัฒนาโดย Meta (Facebook) ช่วยให้ผู้พัฒนาสามารถสร้างแอปพลิเคชันสำหรับ iOS และ Android ได้พร้อมกันด้วยการเขียนโค้ดภาษา JavaScript และเฟรมเวิร์ก React แทนที่จะต้องแยกกันเขียนด้วย Swift/Objective-C (iOS) และ Kotlin/Java (Android) (React Native, 2024)

• **Native Performance:** แตกต่างจาก Hybrid App ยุคเก่าที่รันผ่าน Web View เพราะ React Native จะแปลงคอมโพเนนต์ JavaScript ไปเป็นคอมโพเนนต์ Native จริง ๆ ของอุปกรณ์ ทำให้ได้ประสิทธิภาพที่ไหลลื่นและรูปลักษณ์ที่เป็นธรรมชาติ

• **Cross-Platform:** เขียนโค้ดชุดเดียว (Single Codebase) สามารถแชร์การทำงานได้มากกว่า 80-90% ระหว่างแพลตฟอร์ม

Expo คืออะไร?

Expo คือชุดเครื่องมือ (Suite of Tools) และบริการเสริมที่ครอบทับอยู่บน React Native อีกชั้นหนึ่ง เพื่อช่วยลดความซับซ้อนในการตั้งค่าและเขียนโค้ด Native โดยในปัจจุบัน Expo เป็นเครื่องมือมาตรฐานที่ทางทีมงาน React Native แนะนำอย่างเป็นทางการในการเริ่มต้นโครงการใหม่ (Expo, 2024)

หมายเหตุ (Note): Expo SDK 54 นำเสนอฟีเจอร์ความปลอดภัย ประสิทธิภาพที่สูงขึ้น การใช้ React 19 และมาพร้อมกับโครงสร้างการจัดเส้นทางโฟลเดอร์แบบไฟล์ (File-based Routing) เวอร์ชันล่าสุดผ่าน **Expo Router v6** (Expo, 2024)

1.2 การเปรียบเทียบการเลือกพัฒนาแอปพลิเคชัน

ก่อนเริ่มต้น เรามาทำความเข้าใจแนวทางในการรันแอปพลิเคชัน 3 แบบหลัก ๆ:

ตารางที่ 1.1 การเปรียบเทียบแนวทางการพัฒนาแอปพลิเคชันด้วย React Native และ Expo

หัวข้อเปรียบเทียบ	Expo Go	Expo	React Native CLI

		Development Builds	
ความง่ายในการเริ่ม	ง่ายที่สุด (ติดตั้งแอปจาก Store)	ปานกลาง (ต้องติดตั้ง SDK ท้องถิ่น)	ยากที่สุด (ต้องตั้งค่า SDK และตัวคอมไพล์)
โค้ดภาษา Native	ใช้ได้เฉพาะคอมโพเนนต์มาตรฐานที่แถมมา	เขียนโค้ด Native (Swift/Kotlin) เสริมได้	เขียนโค้ด Native เสริมได้อย่างอิสระ
การสร้างและติดตั้ง	รันผ่านแอปแวดล้อมแชร์ร่วมกัน	สร้างไฟล์แอปเฉพาะกิจ (.apk / .ipa)	ต้องคอมไพล์ทุกครั้งที่ทดสอบโค้ด
เหมาะสำหรับ	ผู้เริ่มต้น, โปรเจคขนาดเล็ก, ทำ UI	โปรเจคจริงขนาดกลาง-ใหญ่, ใช้ปลั๊กอินภายนอก	โปรเจคขนาดใหญ่ที่ต้องการควบคุมระดับ OS

สำหรับหลักสูตรนี้ เราจะเริ่มต้นด้วย Expo Go เพื่อให้ผู้เรียนได้ทดลองเขียนและรันแอปบนอุปกรณ์จริงอย่างรวดเร็วที่สุด จากนั้นจะขยายไปสู่หัวข้อสร้างแอปพลิเคชันแบบกำหนดเองในบทหลัง

1.3 การเตรียมความพร้อมและติดตั้งสภาพแวดล้อม (Environment Setup)

เพื่อให้เครื่องคอมพิวเตอร์ของคุณพร้อมสำหรับการพัฒนาแอปพลิเคชัน ให้ติดตั้งโปรแกรมต่าง ๆ ตามขั้นตอนดังนี้:

ขั้นที่ 1: ติดตั้ง Node.js

Node.js เป็นโปรแกรมจำเป็นสำหรับรันสคริปต์ JavaScript และตัวติดตั้งแพ็กเกจ (npm)

1. ไปที่เว็บไซต์ Node.js Official Website (Node.js, 2024)
2. ดาวน์โหลดและติดตั้งเวอร์ชัน **LTS** (แนะนำเวอร์ชัน 20 หรือ 22 ขึ้นไป)
3. ตรวจสอบความสำเร็จของการติดตั้งใน Terminal/PowerShell:

```
node -v
npm -v
```

ขั้นที่ 2: ติดตั้ง Visual Studio Code (VS Code)

VS Code เป็นโปรแกรมเขียนโค้ดและแก้ไขข้อความยอดนิยม

1. ดาวน์โหลดที่ VS Code Website (Microsoft, 2024)

2. ติดตั้งส่วนขยาย (Extensions) ที่แนะนำสำหรับการเขียน React Native:

- **ES7+ React/Redux/React-Native snippets** (ช่วยคีย์ลัดสร้างโค้ด)
- **Prettier - Code formatter** (ช่วยจัดรูปแบบโค้ดอัตโนมัติ)
- **Expo Tools** (ช่วยตรวจสอบไฟล์ `app.json`)

ขั้นที่ 3: ติดตั้ง Git

ใช้สำหรับการจัดการเวอร์ชันโค้ด (Version Control) ดาวน์โหลดได้จาก Git SCM (Git, 2024)

ขั้นที่ 4: ติดตั้งแอปพลิเคชัน Expo Go บนสมาร์ทโฟน

1. **iOS:** ดาวน์โหลดแอปพลิเคชัน **Expo Go** จาก App Store
2. **Android:** ดาวน์โหลดแอปพลิเคชัน **Expo Go** จาก Google Play Store
3. ตรวจสอบให้มั่นใจว่าสมาร์ทโฟนและเครื่องคอมพิวเตอร์ที่ใช้เขียนโค้ด เชื่อมต่อวงเครือข่ายอินเทอร์เน็ต (Wi-Fi) วงเดียวกัน

1.4 การสร้างโปรเจคใหม่ด้วย create-expo-app

เปิด Terminal (ใน macOS/Linux) หรือ PowerShell (ใน Windows) และย้ายไดเรกทอรีไปยังตำแหน่งที่ต้องการสร้างโปรเจค จากนั้นรันคำสั่ง:

```
npx create-expo-app@latest my-first-app
```

- `npx create-expo-app@latest`: คำสั่งดาวน์โหลดสคริปต์สร้างแอปเวอร์ชันล่าสุด
- `my-first-app`: ชื่อโปรเจคและโฟลเดอร์ที่เราต้องการสร้าง

เมื่อการดาวน์โหลดเสร็จสมบูรณ์ ให้ใช้คำสั่งเปลี่ยนโฟลเดอร์เข้าไปในโปรเจค:

```
cd my-first-app
```

การรีเซ็ตโปรเจคเริ่มต้น (Reset Project Layout)

สำหรับเทมเพลตยุคใหม่ของ Expo ที่สร้างผ่านคำสั่งเริ่มต้น หากผู้เรียนไม่ต้องการหน้าจอตัวอย่างหรือ UI ที่ติดมากับระบบ (เช่น แถบ Tabs ด้านล่าง และหน้าจอ Explore) ทาง Expo มีสคริปต์ล้างโครงสร้างเพื่อให้คุณเริ่มต้นทำแอปจากศูนย์ได้อย่างรวดเร็ว (Expo, 2024):

```
npm run reset-project
```

ข้อควรจำ (Important): คำสั่งนี้จะทำการย้ายโค้ดเทมเพลตเริ่มต้นทั้งหมดไปเก็บไว้ที่โฟลเดอร์ `app-example/` เพื่อใช้ในการศึกษาภายหลัง และระบบจะทำการสร้างโฟลเดอร์ `app/`

ขึ้นมาใหม่พร้อมไฟล์ index.tsx แบบเปล่า ๆ เพื่อให้เราพร้อมเขียนหน้าจอแอปพลิเคชันจากศูนย์ทันที

1.5 โครงสร้างของโปรเจค React Native Expo SDK 54

เมื่อเราเปิดโฟลเดอร์โปรเจคด้วย VS Code จะพบโครงสร้างไฟล์หลัก ๆ ดังนี้:

```
my-first-app/
├── app/                    # โฟลเดอร์สำหรับเก็บโค้ดหน้าจอและการนำทาง (Expo Router)
│   ├── _layout.tsx         # เลย์เอาต์หลักครอบทุกหน้าจอ
│   └── index.tsx           # หน้าจอแรกสุดของแอปพลิเคชัน (Home Screen)
├── assets/                 # ที่เก็บรูปภาพ ฟอนต์ และสัญลักษณ์ของแอป
├── app.json                # ไฟล์การตั้งค่าคุณสมบัติแอปพลิเคชัน (App Metadata)
├── package.json            # รายชื่อ Library และชุดคำสั่งสคริปต์ในการรันแอป
├── tsconfig.json           # การตั้งค่าพฤติกรรมสำหรับภาษา TypeScript
└── node_modules/           # โฟลเดอร์ที่เก็บแพ็กเกจ Library ทั้งหมด (ห้ามแก้ไขโดยตรง)
```

รายละเอียดไฟล์ตั้งค่าที่สำคัญ:

1. `app.json`: ใช้กำหนดชื่อแอปพลิเคชัน โลโก้ (icon) ภาพตอนเข้าแอป (splash image) สิทธิ์การเข้าถึงอุปกรณ์ (permissions) และเวอร์ชันของแอป

2. `app/index.tsx`: จุดเริ่มต้นการเขียนหน้าตาแอปพลิเคชัน เช่น:

```tsx
import { StyleSheet, Text, View } from 'react-native';

export default function HomeScreen() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Hello, Expo SDK 54! 👋</Text>
      <Text style={styles.subtitle}>นี่คือแอปพลิเคชันแรกของฉัน</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a', // สีพื้นหลัง Slate เข้มสไตล์โมเดิร์น
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#f8fafc',
    marginBottom: 8,
  },
```

```
  subtitle: {
    fontSize: 16,
    color: '#94a3b8',
  },
});
```

1.6 การรันโปรเจคและการทดสอบแอปพลิเคชัน

ในโฟลเดอร์โปรเจค ให้รันคำสั่งสตาร์ทเซิร์ฟเวอร์นักพัฒนา:

```
npm start
```

หรือรันผ่านคำสั่งเฉพาะ:

```
npx expo start
```

ตัวจัดการการรัน (Expo CLI) จะเริ่มต้นเซิร์ฟเวอร์ **Metro Bundler** และแสดงหน้าอินเทอร์เฟซควบคุมขึ้นมา รวมถึงสร้าง **QR Code** บนหน้าจอคอนโซล (Expo, 2024):

```
› Metro waiting on exp://192.168.1.33:8081
› Scan the QR code above with Expo Go (Android) or the Camera app (iOS)

› Press a │ open on Android
› Press i │ open on iOS simulator
› Press w │ open on web
› Press r │ reload app
```

ขั้นตอนการดูผลลัพธ์บนอุปกรณ์ต่าง ๆ:

A. สำหรับสมาร์ทโฟนส่วนตัว (แนะนำ)

1. **ระบบ Android:** เปิดแอป **Expo Go** -> แตะเมนู "Scan QR Code" -> สแกนโค้ดบนคอมพิวเตอร์ของคุณ

2. **ระบบ iOS:** เปิดแอป **Camera** (กล้องมาตรฐาน) -> ส่อง QR Code -> แตะลิงก์เปิดด้วยแอป Expo Go

3. เมโทรบันด์เลอร์จะทำงานส่งโค้ด JavaScript มาแปลงร่างเป็นคอมโพเนนต์ Native แสดงผลบนหน้าจอมือถือของคุณทันที!

B. สำหรับตัวจำลองคอมพิวเตอร์ (Simulators/Emulators)

• **iOS Simulator:** ต้องติดตั้ง Xcode (เฉพาะ macOS) จากนั้นกดปุ่ม `i` บนคีย์บอร์ดใน Terminal

• **Android Emulator:** ต้องติดตั้ง Android Studio และสร้าง Virtual Device (AVD) จากนั้นกดปุ่ม **a** บนคีย์บอร์ดใน Terminal

เคล็ดลับ (Tip): Fast Refresh: เมื่อคุณทำการแก้ไขข้อความใด ๆ ในไฟล์ **app/index.tsx** แล้วกดบันทึก (Save) หน้าจอแอปพลิเคชันบนมือถือหรือตัวจำลองของคุณจะอัปเดตผลลัพธ์ใหม่ทันทีในเวลาไม่ถึงวินาทีโดยไม่ต้องเริ่มรันแอปใหม่ทั้งหมด

1.7 สรุปท้ายบทเรียน

ทำความรู้จัก React Native และ Expo: React Native เป็นเครื่องมือพัฒนาแอปพลิเคชันแบบครอสแพลตฟอร์มที่ใช้ JavaScript/React ในการควบคุมคอมโพเนนต์ Native จริงของอุปกรณ์เพื่อให้ได้ประสิทธิภาพสูง โดยทำงานร่วมกับ Expo SDK 54 ซึ่งเป็นชุดเครื่องมืออำนวยความสะดวกในการจัดสรรสถาปัตยกรรมและไลบรารีมาตรฐานระดับระบบปฏิบัติการอย่างครบถ้วน

การเปรียบเทียบการเลือกพัฒนาแอปพลิเคชัน: แนวทางการพัฒนาแอปพลิเคชันแบ่งออกเป็น 3 รูปแบบหลัก ได้แก่ Expo Go ซึ่งติดตั้งและเริ่มรันได้ง่ายที่สุดเหมาะแก่การศึกษาและทดลองใช้เครื่องมือมาตรฐาน, Expo Development Builds ที่เหมาะสมสำหรับแอปพลิเคชันขนาดกลางถึงใหญ่เนื่องจากรองรับการปรับแต่งเนทีฟโมดูลเพิ่มเติมได้เอง และ React Native CLI ที่ให้ความอิสระและระดับสิทธิ์การควบคุมระดับลึกที่สุดแก่ผู้พัฒนารวมถึงการเขียนโค้ด Native โดยตรง

การเตรียมความพร้อมและติดตั้งสภาพแวดล้อม: การเริ่มต้นพัฒนาแอปพลิเคชันต้องการสภาพแวดล้อมการเขียนโค้ดที่ประกอบไปด้วย Node.js เวอร์ชัน LTS สำหรับการรัน JavaScript นอกเบราว์เซอร์, VS Code พร้อมส่วนขยายอำนวยความสะดวก เช่น Expo Tools และ Prettier, ระบบควบคุมเวอร์ชันด้วย Git และแอปพลิเคชัน Expo Go บนระบบปฏิบัติการ iOS หรือ Android เพื่อทำหน้าที่แปลงผลและรันแอปบนอุปกรณ์จริงในวงเน็ตเวิร์กเดียวกัน

การสร้างโปรเจคใหม่ด้วย create-expo-app: การสร้างโครงสร้างโฟลเดอร์สำหรับแอปพลิเคชันใหม่ใช้คำสั่ง **npx create-expo-app@latest** ตามด้วยชื่อโปรเจกต์ ซึ่งจะได้โครงสร้างการทำงานแบบสมบูรณ์พร้อมใช้ และผู้พัฒนาสามารถรันคำสั่ง **npm run reset-project** เพื่อล้างหน้าต่างจำลองของเทมเพลตเริ่มต้นและปรับโครงสร้างโปรเจกต์ให้เหลือเพียงหน้ากระดาษเปล่าสำหรับการเขียนแอปพลิเคชันตั้งแต่เริ่มต้น

โครงสร้างของโปรเจค React Native Expo SDK 54: โฟลเดอร์ของโปรเจกต์ที่สร้างขึ้นจะแบ่งสัดส่วนการทำงานออกเป็น โฟลเดอร์ **app/** สำหรับเก็บหน้าจอและระบบการเปลี่ยนหน้าผ่านไฟล์ (Expo Router), โฟลเดอร์ **assets/** เก็บภาพไอคอนและส่วนประกอบด้านมีเดีย, ไฟล์ **app.json** ทำหน้าที่ควบคุมการตั้งค่าคุณสมบัติ (Metadata) ของแอป และไฟล์ **app/index.tsx** ซึ่งเป็นคอมโพเนนต์แรกที่แอปเรียกมาแสดงผลและทำหน้าที่เป็นจุดเริ่มต้นของโค้ดหลัก

การรันโปรเจคและการทดสอบแอปพลิเคชัน: การรันแอปทำได้โดยใช้คำสั่ง `npm start` หรือ `npx expo start` เพื่อเริ่มการทำงานของ Metro Bundler ซึ่งจะทำการรวบรวมไฟล์เขียนโค้ดและส่งผลผ่าน QR Code ให้สแกนดูพฤติกรรมจริงผ่านแอป Expo Go บนอุปกรณ์สมาร์ทโฟน หรือทดสอบผ่านโปรแกรมจำลอง Emulator/Simulator โดยมีการสนับสนุนฟีเจอร์ Fast Refresh ที่ช่วยสะท้อนผลการแก้ไขโค้ดสู่หน้าจอทดสอบโดยทันที

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้และความเข้าใจเกี่ยวกับความรู้พื้นฐานการตั้งค่าระบบ และโครงสร้างเบื้องต้นของ React Native Expo SDK 54

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดอธิบายความแตกต่างที่สำคัญที่สุดระหว่าง React Native และเฟรมเวิร์กประเภท Hybrid App ยุคเก่า (เช่น Cordova หรือ PhoneGap) ได้ถูกต้องที่สุด?

ก. React Native เขียนด้วยภาษา Swift แต่ Hybrid App เขียนด้วยภาษา HTML/CSS

ข. React Native จะแปลงคอมโพเนนต์ไปเป็น Native UI component จริง ๆ ของอุปกรณ์ ส่วน Hybrid App จะแสดงผลผ่าน Web View

ค. React Native รันได้เฉพาะบนระบบปฏิบัติการ iOS เท่านั้น ในขณะที่ Hybrid App รันได้ทุกระบบ

ง. React Native ไม่สามารถเชื่อมต่อกับฮาร์ดแวร์ของอุปกรณ์ได้ แต่ Hybrid App ทำได้

ข้อ 2: เมื่อเราใช้เทมเพลตเริ่มต้นของ Expo SDK 54 และรันคำสั่ง `npm run reset-project` จะเกิดผลลัพธ์ใดขึ้นกับโครงการ?

ก. ลบการเชื่อมต่อเครือข่ายของแอปและล้างข้อมูลแคชทั้งหมด

ข. ลบโปรเจคทิ้งอย่างถาวรและสร้างโฟลเดอร์เปล่าขึ้นมาใหม่

ค. ย้ายไฟล์ตัวอย่างและโครงสร้างเดิมไปไว้ในโฟลเดอร์ `app-example/` และสร้างโฟลเดอร์ `app/` ใหม่พร้อมไฟล์เริ่มต้นที่ว่างเปล่า

ง. ทำการคอมไพล์โปรเจคให้กลายเป็นไฟล์สำหรับติดตั้ง (.apk และ .ipa) ทันที

ข้อ 3: ข้อจำกัดที่สำคัญที่สุดของการใช้เครื่องมือ "Expo Go" ในการทดสอบแอปพลิเคชันคืออะไร?

ก. ไม่สามารถทดสอบบนสมาร์ทโฟนระบบปฏิบัติการ iOS ได้

ข. ไม่สามารถทดสอบโค้ดขณะแก้ไขแบบเรียลไทม์ได้ ต้องคอมไพล์ใหม่ทุกครั้ง

ค. ไม่สามารถทดสอบแอปพลิเคชันที่ติดตั้งไลบรารีหรือเขียนโค้ดภาษา Native เสริมเองภายนอกมาตรฐานของ Expo ได้

ง. ต้องเสียค่าบริการรายเดือนในการสแกน QR Code เพื่อเปิดใช้งาน

ข้อ 4: ไฟล์ใดในโปรเจค React Native Expo ที่ทำหน้าที่เก็บข้อมูลคุณสมบัติแอปพลิเคชัน (App Metadata) เช่น ชื่อแอป โลโก้ (Icon) หน้าต้อนรับ (Splash Screen) และเวอร์ชันของแอป?

ก. `package.json`

ข. `app.json`

ค. `tsconfig.json`

ง. `_layout.tsx`

ข้อ 5: คุณสมบัติ "Fast Refresh" ในสภาพแวดล้อมการพัฒนาของ Expo ทำหน้าที่อะไร?

ก. ช่วยเพิ่มความเร็วในการดาวน์โหลดไลบรารีจากเซิร์ฟเวอร์ npm

ข. อัปเดตอินเทอร์เฟซของแอปบนอุปกรณ์ทดสอบทันทีที่กดบันทึกโค้ด (Save) โดยไม่ต้องเริ่มแอปพลิเคชันใหม่ทั้งหมด

ค. ช่วยเคลียร์แคชและรีสตาร์ตระบบปฏิบัติการของมือถือโดยอัตโนมัติ

ง. การอัปเดตแอปพลิเคชันขึ้นสู่ Google Play Store และ App Store อย่างรวดเร็ว

ข้อ 6: คำสั่งมาตรฐานใดที่แนะนำให้ใช้รันใน Terminal เพื่อเปิดเซิร์ฟเวอร์นักพัฒนา Metro Bundler ของ Expo?

ก. `npm run ios`

ข. `npx expo run:android`

ค. `npx expo start`

ง. `npx react-native start`

ข้อ 7: หากหน้าจอแอปพลิเคชันบนตัวจำลอง (Simulator) ไม่ตอบสนอง หรือ Fast Refresh ไม่ทำงาน คุณสามารถกดปุ่มใดบนคีย์บอร์ดใน Terminal ของเซิร์ฟเวอร์นักพัฒนาเพื่อสั่งโหลดหน้าจอแอปใหม่ด้วยตนเอง (Manual Reload)?

ก. กดปุ่ม `r`

ข. กดปุ่ม `i`

ค. กดปุ่ม `a`

ง. กดปุ่ม `d`

ข้อ 8: การทดสอบแอปพลิเคชันด้วยสมาร์ทโฟนจริงผ่านแอป Expo Go มีเงื่อนไขการเชื่อมต่อเครือข่ายอินเทอร์เน็ตที่จำเป็นอย่างไร?

ก. สมาร์ทโฟนต้องเชื่อมต่อ Cellular 5G ส่วนเครื่องคอมพิวเตอร์เชื่อมต่อ Wi-Fi บ้าน

ข. ทั้งสมาร์ทโฟนและเครื่องคอมพิวเตอร์ต้องเชื่อมต่อผ่านสาย USB ตลอดเวลาเท่านั้น

ค. สมาร์ทโฟนและเครื่องคอมพิวเตอร์ต้องเชื่อมต่อในเครือข่ายวงแลนเดียวกัน (เช่น Wi-Fi วงเดียวกัน)

ง. เครื่องคอมพิวเตอร์และมือถือต้องเปิดสัญญาณ Bluetooth เชื่อมต่อกันก่อน

ข้อ 9: โครงสร้างโฟลเดอร์ของ Expo SDK 54 ที่ใช้ระบบนำทางตามโครงสร้างไฟล์ (File-based Routing) จะมองหาไฟล์เริ่มต้นสำหรับหน้าหลักของแต่ละโฟลเดอร์จากชื่อไฟล์ใด?

ก. `App.tsx`

ข. `index.tsx`

ค. `main.tsx`

ง. `root.tsx`

ข้อ 10: ส่วนขยายใดใน VS Code ที่เป็นตัวช่วยจัดระเบียบและจัดรูปแบบความสวยงามของโค้ดให้ถูกต้องตามมาตรฐานโดยอัตโนมัติทุกครั้งเมื่อกดบันทึกไฟล์?

ก. Prettier - Code formatter

ข. ES7+ React/Redux/React-Native snippets

ค. Expo Tools

ง. GitLens

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: ทั้ง React Native และ Hybrid App ในแง่เขียนโค้ดจะใช้เทคโนโลยีเว็บ (โดยเฉพาะ JavaScript/TypeScript) ไม่ใช่เขียนด้วย Swift เป็นหลัก

• ข. ถูก: นี่คือคุณลักษณะเด่นของ React Native ที่ทำให้ประสิทธิภาพ (Performance) ใกล้เคียงกับ Native App แท้ ๆ เพราะคอมโพเนนต์อย่าง `<View>` หรือ `<Text>` จะถูกจับคู่และสร้างเป็นคอมโพเนนต์ระดับ OS จริง ๆ ของมือถือตัวนั้น ส่วน Hybrid App ยุคเก่าจะรันเว็บแอปจำลองภายในเบราว์เซอร์ครอบ (Web View)

• ค. ผิด: React Native เป็น Cross-platform สามารถรันได้ทั้ง iOS, Android และ Web

• **ง. ผิด:** React Native มีความสามารถสูงในการเข้าถึงฮาร์ดแวร์ผ่าน API ส่วนติดต่อของภาษา Native

เฉลย ข้อ 2: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** คำสั่งนี้ไม่เกี่ยวข้องกับการล้างแคชหรือลบข้อมูลเครื่องจำลอง

• **ข. ผิด:** โปรเจคและไดเรกทอรีหลักยังคงอยู่ไม่ได้ถูกลบทั้งหมด

• **ค. ถูก:** การรัน `npm run reset-project` จะรันสคริปต์เพื่อแยกโค้ดตัวอย่างของระบบ (เช่น explore screen หรือ components ต่าง ๆ) ไปเก็บไว้ที่ `app-example/` และสร้างโฟลเดอร์ `app/` สำหรับเขียนโค้ดที่สะอาดใหม่ขึ้นมา เพื่อเริ่มต้นโครงสร้างของคุณเอง

• **ง. ผิด:** การคอมไพล์โปรเจคเป็นไฟล์ติดตั้งทำโดยใช้บริการของ EAS Build ไม่ใช่สคริปต์ตัวนี้

เฉลย ข้อ 3: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** Expo Go รองรับทั้ง iOS และ Android อย่างสมบูรณ์

• **ข. ผิด:** Expo Go รองรับ Fast Refresh สามารถอัปเดตหน้าจอแบบเรียลไทม์ได้ทันที

• **ค. ถูก:** เนื่องจาก Expo Go เป็นแอปสำเร็จรูปที่มีไลบรารีมาตรฐานติดตั้งมาให้แล้ว หากคุณต้องการเขียนโค้ด Native (เช่น Swift, Kotlin) หรือใช้แพ็กเกจที่จำเพาะเจาะจงที่แก้ไขส่วนประกอบ OS ดั้งเดิม คุณจะต้องเปลี่ยนไปรันผ่าน Expo Development Builds แทน

• **ง. ผิด:** เครื่องมือ Expo Go และการแชร์หน้าจอผ่าน Metro Bundler ใช้งานได้ฟรีไม่มีค่าใช้จ่าย

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `package.json` มีหน้าที่เก็บรายชื่อ Dependencies และสคริปต์คำสั่งรันระบบ

• **ข. ถูก:** `app.json` เป็นไฟล์คอนฟิกูเรชันหลักของ Expo (Expo config) ที่เอาไว้ตั้งค่าข้อมูลทั่วไปของแอปพลิเคชัน โครงสร้างสีเริ่มต้น รูปภาพไอคอน ภาพหน้าต้อนรับ และการกำหนดค่าเฉพาะของแพลตฟอร์มปลายทาง

• **ค. ผิด:** `tsconfig.json` เป็นไฟล์สำหรับตั้งค่าคอมไพเลอร์ของภาษา TypeScript

• **ง. ผิด:** `_layout.tsx` เป็นไฟล์กำหนดโครงสร้างการนำทางและการแสดงผลของหน้าจอ (Layout) ของ Expo Router

เฉลย ข้อ 5: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: การจัดการแพ็กเกจไลบรารีเป็นหน้าที่ของ npm / yarn

• ข. ถูก: Fast Refresh เป็นฟีเจอร์เด่นของการพัฒนาแอปพลิเคชันด้วย React Native ที่จะคอยเฝ้าดูการเปลี่ยนแปลงของไฟล์โค้ด (Watch Mode) เมื่อมีการบันทึกไฟล์ โค้ดที่เปลี่ยนจะถูกส่งไปทำงานบนเครื่องที่เปิดจำลองแอปอยู่ทันที ช่วยลดระยะเวลาที่นักพัฒนาต้องรอรันระบบใหม่ลงอย่างมหาศาล

• ค. ผิด: ฟังก์ชันนี้ไม่มีความสามารถในการยุ่งเกี่ยวกับระบบปฏิบัติการหลักของสมาร์ทโฟน

• ง. ผิด: การอัปโหลดแอปพลิเคชันขึ้นสโตร์ใช้ EAS Submit หรือส่งไฟล์ Build โดยตรง

เฉลย ข้อ 6: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: `npm run ios` ใช้สำหรับสั่งรันแอปบน iOS Simulator ทันที (ซึ่งจะสตาร์ท Metro ให้อยู่แล้ว แต่เป็นคำสั่งจำเพาะเจาะจงแพลตฟอร์ม)

• ข. ผิด: `npx expo run:android` ใช้สำหรับคอมไพล์โปรเจคแบบ Development Builds รันบนเครื่องจำลอง Android ซึ่งช้ากว่าและยุ่งยากกว่าการทดสอบด้วย Expo Go ในขั้นเริ่มต้น

• ค. ถูก: `npx expo start` (หรือ alias `npm start` ใน package.json) คือคำสั่งพื้นฐานและแนะนำที่สุดของ Expo ในการเปิดระบบเซิร์ฟเวอร์ Metro Bundler และสร้าง QR Code เพื่อสแกนรันแอป

• ง. ผิด: `npx react-native start` เป็นคำสั่งรันเซิร์ฟเวอร์สำหรับโปรเจคที่เป็น React Native CLI บริสุทธิ์

เฉลย ข้อ 7: ตอบ ก.

• เหตุผลประกอบ:

• ก. ถูก: ในหน้าต่าง Terminal ของเซิร์ฟเวอร์ Metro การกดปุ่ม `r` ย่อมาจาก **Reload** จะส่งสัญญาณบอกแอปทดสอบบนเครื่องจำลองหรือโทรศัพท์จริงให้รีโหลดหน้าจอใหม่ทั้งหมดแบบแมนนวล

• ข. ผิด: การกดปุ่ม `i` ย่อมาจาก **iOS** ใช้สำหรับสั่งเปิดแอปบน iOS Simulator

• ค. ผิด: การกดปุ่ม `a` ย่อมาจาก **Android** ใช้สำหรับสั่งเปิดแอปบน Android Emulator

• **ง. ผิด:** การกดปุ่ม `d` ย่อมาจาก **Dev Menu** ใช้สำหรับสั่งเปิดเมนูนักพัฒนา (Developer Menu) บนแอปจำลอง

เฉลย ข้อ 8: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** การเปิดข้ามเครือข่ายผ่าน Cellular อาจทำให้แอปหาเครื่องคอมพิวเตอร์ในวงแลนของเครือข่ายนักพัฒนาไม่เจอ ยกเว้นมีการตั้งค่าอุโมงค์การเชื่อมต่อแบบ Tunnel (ซึ่งทำได้ยากและช้ากว่าปกติมาก)

• **ข. ผิด:** ไม่จำเป็นต้องต่อสาย USB เสมอไปเมื่อใช้งาน Expo Go เพราะระบบสามารถส่งผ่านข้อมูลทาง Wi-Fi วงแลนได้แบบไร้สาย

• **ค. ถูก:** หัวใจสำคัญของการสื่อสารระหว่าง Metro Bundler บน PC และมือถือที่ใช้รันแอปคือต้องอยู่ในเครือข่ายภายในบ้าน/ที่ทำงานเดียวกัน (Subnet วงเดียวกัน) เพื่อให้เครื่องมือถือสามารถค้นหา IP Address และ Port 8081 ของ PC พบ

• **ง. ผิด:** Expo CLI และ Expo Go ไม่สื่อสารโค้ดโปรแกรมหลักผ่านบลูทูธ

เฉลย ข้อ 9: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `App.tsx` เป็นไฟล์เริ่มต้นของโปรเจค React Native แบบดั้งเดิม แต่ไม่ใช่โครงสร้างของ Expo Router ที่เปิดตัวขึ้นมาในรุ่นหลัง

• **ข. ถูก:** โครงสร้าง File-based Routing (Expo Router) จะทำงานคล้ายกับการจัดการหน้าเว็บ (Web Router) โดยไฟล์ที่ทำหน้าที่เป็นจุดเริ่มต้นหรือหน้าแรกของโฟลเดอร์ใด ๆ จะต้องตั้งชื่อว่า `index.tsx` (หรือ `index.js`, `index.ts`) เสมอ

• **ค. ผิด:** `main.tsx` ไม่เกี่ยวข้องกับการหาเส้นทางเริ่มต้นของโฟลเดอร์

• **ง. ผิด:** `root.tsx` ไม่ได้ใช้ระบุตำแหน่งของ Layout เริ่มต้นในระดับโฟลเดอร์

เฉลย ข้อ 10: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก: Prettier** คือตัวจัดการปรับสไตล์โค้ด (Code Formatter) ยอดนิยมที่จะจัดย่อหน้า แทรกเครื่องหมาย semicolon หรือจัดระเบียบวงเล็บปีกกาและช่องว่างโดยอัตโนมัติเมื่อกดจัดรูปแบบหรือเซฟไฟล์

• **ข. ผิด: ES7+ React/Redux/React-Native snippets** เป็นเพียงตัวช่วยสร้างโครงโค้ดแบบรวดเร็ว (เช่น พิมพ์ `rfc` เพื่อสร้าง Component) แต่ไม่ได้มีหน้าที่เป็นตัวจัดรูปแบบโค้ด

• **ค. ผิด: Expo Tools** ช่วยทำไฮไลต์ไวยากรณ์ในไฟล์คอนฟิก `app.json` และช่วยทำ Auto-complete ในบางจุดที่เกี่ยวข้องกับเครื่องมือของ Expo เท่านั้น

• **ง. ผิด: GitLens** เป็นตัวเสริมสำหรับการแสดงประวัติและเจ้าของบรรทัดโค้ดบนระบบ Git

ปฏิบัติการท้ายบทเรียน: การเตรียมสภาพแวดล้อมและการสร้างแอปพลิเคชันแรก

ปฏิบัติการนี้จะช่วยให้ผู้เรียนได้ลงมือทำจริงตั้งแต่ขั้นตอนการติดตั้งโปรแกรมพื้นฐาน การสร้างโปรเจค React Native Expo SDK 54 การรีเซ็ตโครงสร้างตัวอย่างจากเทมเพลตเริ่มต้น การเขียนโค้ดภาษา TypeScript/JSX ตัวแรก และทดลองรันแอปพลิเคชันจริงผ่านเซิร์ฟเวอร์นักพัฒนา

1. วัตถุประสงค์การเรียนรู้

1. สามารถติดตั้งเครื่องมือพัฒนาแอปพลิเคชันมือถือ (Node.js, VS Code) และกำหนดสภาพแวดล้อมการรันได้อย่างถูกต้อง
2. สามารถสร้างโปรเจค React Native Expo ตัวใหม่ผ่านเทมเพลต TypeScript ได้สำเร็จ
3. สามารถใช้สคริปต์ `reset-project` เพื่อล้างโค้ดเทมเพลตตัวอย่างและจัดโครงสร้างโปรเจคเริ่มต้นใหม่ได้
4. สามารถเขียนคอมโพเนนต์พื้นฐานเพื่อสร้างหน้าจออินเทอร์เฟซต้อนรับอย่างง่ายได้ด้วยตนเอง
5. สามารถใช้งาน Metro Bundler และเปิดทดสอบหน้าจอแอปพลิเคชันจริงผ่านสมาร์ทโฟน (Expo Go) หรืออุปกรณ์จำลองได้

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนจะต้องดำเนินการเตรียมเครื่องมือและเขียนหน้าต้อนรับ (Welcome Screen) ลงในโปรเจคตามเงื่อนไขดังนี้:

1. ติดตั้ง Node.js และ VS Code พร้อมขยายระบบตัวช่วยโค้ด (Extensions) ที่กำหนด
2. สร้างโปรเจค Expo ใหม่ด้วยชื่อโฟลเดอร์ `my-first-app`
3. ล้างตัวอย่างโปรเจคดั้งเดิมให้เหลือเฉพาะโครงสร้างไฟล์เริ่มต้นเพื่อรอเขียนโปรแกรม
4. หน้าจอหลักต้องแสดงกล่องข้อมูลต้อนรับผู้เรียนซึ่งกึ่งกลางของหน้าจอ (Center alignment)
5. ปรับเปลี่ยนโทนสีพื้นหลังเป็นสไตล์ Slate เข้ม (`#0f172a`) และข้อความทั้งหมดต้องจัดสไตล์ให้อ่านง่าย มีสีสันสอดคล้องกับธีม (สไตล์ Dark Mode)

6. มีส่วนแสดงผล **ชื่อ-นามสกุล** และ **รหัสนักศึกษา/ชื่อผู้เรียน** รวมถึงแสดงสถานะการเชื่อมต่อว่า "Expo SDK 54 - Active"

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งโปรแกรมที่จำเป็น (Software Installation)

1. **ติดตั้ง Node.js:** ไปที่เว็บไซต์ Node.js ดาวน์โหลดรุ่น **LTS** ล่าสุดมาติดตั้งให้เรียบร้อย จากนั้นเช็กความถูกต้องผ่าน Terminal/PowerShell ด้วยคำสั่ง:

```
node -v
npm -v
```

2. **ติดตั้ง Visual Studio Code (VS Code):** ดาวน์โหลดที่ VS Code และเข้าไปที่เมนู Extensions (ไอคอนสี่เหลี่ยมด้านซ้าย) ค้นหาและติดตั้งตัวช่วยต่อไปนี้:

- *ES7+ React/Redux/React-Native snippets*
- *Prettier - Code formatter*
- *Expo Tools*

ขั้นตอนที่ 2: การสร้างโปรเจคใหม่ด้วย create-expo-app

1. เปิด Command Prompt หรือ PowerShell (ใน Windows) หรือ Terminal (ใน macOS/Linux)

2. เปลี่ยนตำแหน่งไดเรกทอรีไปยังโฟลเดอร์ที่คุณต้องการบันทึกงาน เช่น:

```
cd Documents
```

3. รันคำสั่งสร้างโปรเจคใหม่ด้วยคำสั่ง Expo CLI เวอร์ชันล่าสุด:

```
npx create-expo-app@latest my-first-app
```

4. เมื่อสร้างโปรเจคเรียบร้อยแล้ว ให้ย้ายเข้าสู่โฟลเดอร์โปรเจค:

```
cd my-first-app
```

5. เปิดหน้าต่างโปรเจคขึ้นมาบนโปรแกรม VS Code:

```
code .
```

ขั้นตอนที่ 3: ล้างไฟล์เทมเพลตเริ่มต้น (Reset Project Layout)

ในหน้าต่าง VS Code เปิด Terminal ขึ้นมา (กดปุ่มคีย์บอร์ด `Ctrl + ~`) จากนั้นพิมพ์รันสคริปต์เคลียร์โค้ดเริ่มต้น:

```
npm run reset-project
```

สังเกตผลลัพธ์: โฟลเดอร์ `app/` จะถูกล้างส่วนประกอบตัวอย่างออกทั้งหมดและสร้างโครงสร้างที่สะอาดขึ้นมาใหม่ทันที ส่วนโค้ดตัวอย่างเดิมจะถูกสำรองไว้ที่ `app-example/`

ขั้นตอนที่ 4: เขียนโค้ดปรับปรุงหน้าแรก (app/index.tsx)

ให้เปิดไฟล์ `app/index.tsx` ใน VS Code แล้วเขียนโค้ดทับด้วยคอมโพเนนต์หน้าต้อนรับดังต่อไปนี้:

```
import { StyleSheet, Text, View } from 'react-native';

export default function WelcomeScreen() {
  return (
    <View style={styles.container}>
      {/* ส่วนหัวแสดงไอคอนหรือรูปจำลองด้วยอิโมจิ */}
      <Text style={styles.emoji}>🚀</Text>

      {/* หัวข้อหลัก */}
      <Text style={styles.title}>ยินดีต้อนรับเข้าสู่ชั้นเรียน Mobile App</Text>
      <Text style={styles.subtitle}>React Native Expo (SDK 54)</Text>

      {/* การ์ดข้อมูลผู้เรียน */}
      <View style={styles.card}>
        <Text style={styles.cardLabel}>ผู้พัฒนาแอปพลิเคชัน:</Text>
        <Text style={styles.cardName}>สมชาย ยอดรักดี (นายทะเบียน)</Text>
        <Text style={styles.cardId}>รหัสนักศึกษา: 66-12345-6789</Text>
      </View>

      {/* แถบสถานะระบบ */}
      <View style={styles.statusBadge}>
        <Text style={styles.statusText}>● Status: Running on Expo
Go</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a', // สีพื้นหลังเทาดำ Slate เข้ม
    alignItems: 'center',
    justifyContent: 'center',
    padding: 24,
  },
  emoji: {
```

```
    fontSize: 64,
    marginBottom: 16,
  },
  title: {
    fontSize: 22,
    fontWeight: 'bold',
    color: '#38bdf8', // สีฟ้านีออนโดดเด่นสว่างตา
    textAlign: 'center',
    marginBottom: 4,
  },
  subtitle: {
    fontSize: 16,
    color: '#94a3b8', // สีเทาอ่อน สบายตา
    textAlign: 'center',
    marginBottom: 32,
  },
  card: {
    backgroundColor: '#1e293b', // พื้นหลังการ์ดสีเทา Slate
    borderRadius: 16,
    padding: 20,
    width: '100%',
    maxWidth: 320,
    borderWidth: 1,
    borderColor: '#334155', // เส้นขอบสี Slate จาง ๆ
    alignItems: 'flex-start',
    marginBottom: 24,
  },
  cardLabel: {
    fontSize: 12,
    color: '#64748b',
    textTransform: 'uppercase',
    letterSpacing: 1,
    marginBottom: 6,
  },
  cardName: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#f8fafc',
    marginBottom: 4,
  },
  cardId: {
    fontSize: 14,
    color: '#94a3b8',
  },
  statusBadge: {
    backgroundColor: '#064e3b', // พื้นหลังสีเขียวเข้ม
    paddingVertical: 6,
    paddingHorizontal: 12,
    borderRadius: 20,
  },
  statusText: {
    color: '#34d399', // ข้อความสีเขียวพาสเทล
    fontSize: 12,
    fontWeight: '600',
```

```
  },
});
```

ขั้นตอนที่ 5: เปิดเซิร์ฟเวอร์ Metro Bundler

บันทึกไฟล์โค้ด จากนั้นใน Terminal ของ VS Code ให้รันคำสั่งเริ่มทำงานระบบเซิร์ฟเวอร์นักพัฒนา:

```
npx expo start
```

ขั้นตอนที่ 6: สแกนทดสอบบนอุปกรณ์จริง (Expo Go) หรืออุปกรณ์จำลอง

1. เปิดแอปพลิเคชัน **Expo Go** บนมือถือของคุณ (ดาวน์โหลดติดตั้งผ่าน App Store/Play Store ก่อนหน้านี้)

2. **สำหรับ iOS:** เปิดกล้องมือถือสแกน QR Code บนหน้าจอคอมพิวเตอร์เพื่อสลับเปิดรันด้วย Expo Go

3. **สำหรับ Android:** เปิดแอป Expo Go แล้วกดสแกน QR Code ผ่านแอปโดยตรง

4. หากต้องการทดสอบบนเครื่องจำลอง: กดปุ่ม `i` สำหรับ iOS Simulator (สำหรับเครื่อง macOS) หรือกดปุ่ม `a` สำหรับ Android Emulator (ต้องเปิดโปรแกรม Android Virtual Device ก่อนรันคีย์)

ขั้นตอนที่ 7: ทดสอบฟีเจอร์ Fast Refresh

1. เปิดแสดงผลหน้าจอแอปพลิเคชันควบคู่ไปกับหน้าต่างแก้ไขโค้ด VS Code

2. ให้แก้ไขคำว่า "สมชาย ยอดรักดี" ในโค้ด `app/index.tsx` บรรทัดที่ 16 ให้เปลี่ยนเป็น **"ชื่อและนามสกุลจริงของคุณ"**

3. กดปุ่มบันทึกไฟล์ (`Ctrl + S` หรือ `Cmd + S`)

4. สังเกตหน้าจอทดสอบ จะพบว่าข้อความอัปเดตเป็นชื่อของคุณทันทีภายในไม่กี่วินาทีโดยไม่ต้องปิดและรันแอปใหม่

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

โค้ดเฉลยฉบับสมบูรณ์สำหรับปฏิบัติการนี้ถูกระบุไว้เป็นตัวอย่างโครงสร้างใน **ขั้นตอนที่ 4** ผู้เรียนสามารถคัดลอกและบันทึกไฟล์เพื่อรันตรวจสอบความถูกต้องของระบบได้ทันที

เอกสารอ้างอิง

Expo. (2024). *Create an app*. จาก https://docs.expo.dev/versions/v54.0.0/get-started/create-a-project/

Expo. (2024). *Expo Router Introduction*. จาก https://docs.expo.dev/versions/v54.0.0/router/introduction/

Git. (2024). *Git Reference Manual*. จาก https://git-scm.com/doc

Microsoft. (2024). *Visual Studio Code Documentation*. จาก https://code.visualstudio.com/docs

Node.js. (2024). *Node.js Downloads*. จาก https://nodejs.org/

React Native. (2024). *Environment Setup*. จาก https://reactnative.dev/docs/environment-setup